

CAPSTONE PROJECT – REVIEW 2

Smart Home Network: Device Control and Command Delivery Verification

Cisco Packet Tracer • Python Socket Programming • Wireshark

Team Members

RITHIN SAI A— Home LAN Topology Design (Packet Tracer)

ABDUL SAMED K— Device Control Command Program (Python Sockets)

VISHVANANDH M — Command Delivery Verification (Wireshark)

[REGISTERNUMBERS—replace]

Guide:[GUIDENAME—replace]

Table of Contents

- | | | | |
|---|-----------------------------------|----|--------------------------------|
| 1 | Abstract | 8 | Limitations & Future Scope |
| 2 | Introduction | 9 | Ethics & Engineering Standards |
| 3 | Problem Identification & Analysis | 10 | Conference / Paper / Hackathon |
| 4 | Solution Design & Implementation | 11 | Conclusion |
| 5 | Software / Hardware Functionality | 12 | References |
| 6 | Modules Implementation | 13 | Live Demonstration |
| 7 | Results & Comparison | 14 | GPS Photo & Video Evidence |

ABSTRACT

This project implements a complete smart home communication chain and verifies it end to end. A home LAN is designed in Cisco Packet Tracer with a Wi-Fi router, a control PC and four simulated smart devices (light, fan, plug, camera) on the 192.168.1.0/24 network. A Python TCP client-server application then reproduces the control layer: the client sends ON/OFF commands for each device and the server updates the device state and returns a confirmation message. Finally, Wireshark captures the TCP traffic on port 5000 to prove that every command and confirmation actually travelled across the network, and to measure the delivery delay, the number of commands captured and the missed confirmations.

Domain

Computer Networks /
IoT

Approach

Simulate → Program →
Verify

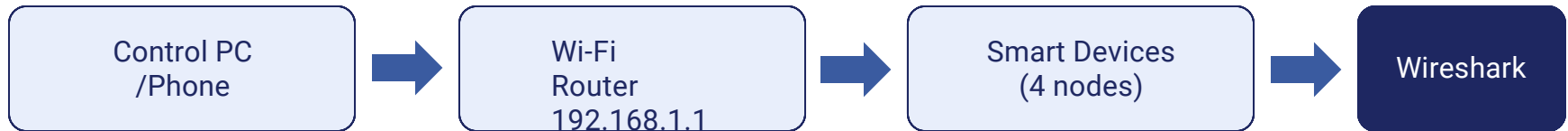
Outcome

Measured, evidence-
backed command
delivery

Introduction

- Smart homes rely on a Wi-Fi network to carry short control messages between a phone/PC and appliances.
- Users judge such a system by two things: does the command reach the device, and is the response immediate?
- This project models that entire path – topology, control software and packet-level verification.

Four devices are controlled: Smart Light, Smart Fan, Smart Plug and Smart Camera, in three student modules.



Problem Identification and Analysis

The Problem

In most smart home setups the user only sees the app screen. If a device does not respond there is no way to tell whether the command was never sent, lost in the network, or ignored by the device.

Analysis of Causes

- Wireless link or addressing errors in the LAN
- Application-layer logic with no acknowledgement
- No instrumentation to observe the actual packets

Why It Matters

Unverified delivery makes faults impossible to localise, hides delay problems, and gives no measurable quality figure for the network.

What Is Required

- A correctly addressed, reachable home LAN
- A request-confirm control protocol
- Packet capture to count and time every message

Solution Design and Implementation

Module 1 -A.RITHINSAI

Home LAN Topology
Design
Tool: Cisco Packet Tracer

- Wireless home gateway .1
- Control-PC .10
- Light .20, Fan .21
- Plug .22, Camera .23
- Mask 255.255.255.0
- Ping test to all devices

Module 2 –K.ABDUL SAMED

Device Control Program
Tool: Python Sockets

- server.py on port 5000
- Device state dictionary
- client.py menu sends

"DEVICE ON/OFF"

- Server replies confirmation
- Records success rate

Module 3 – VISHVANANDH M

Delivery Verification
Tool: Wireshark

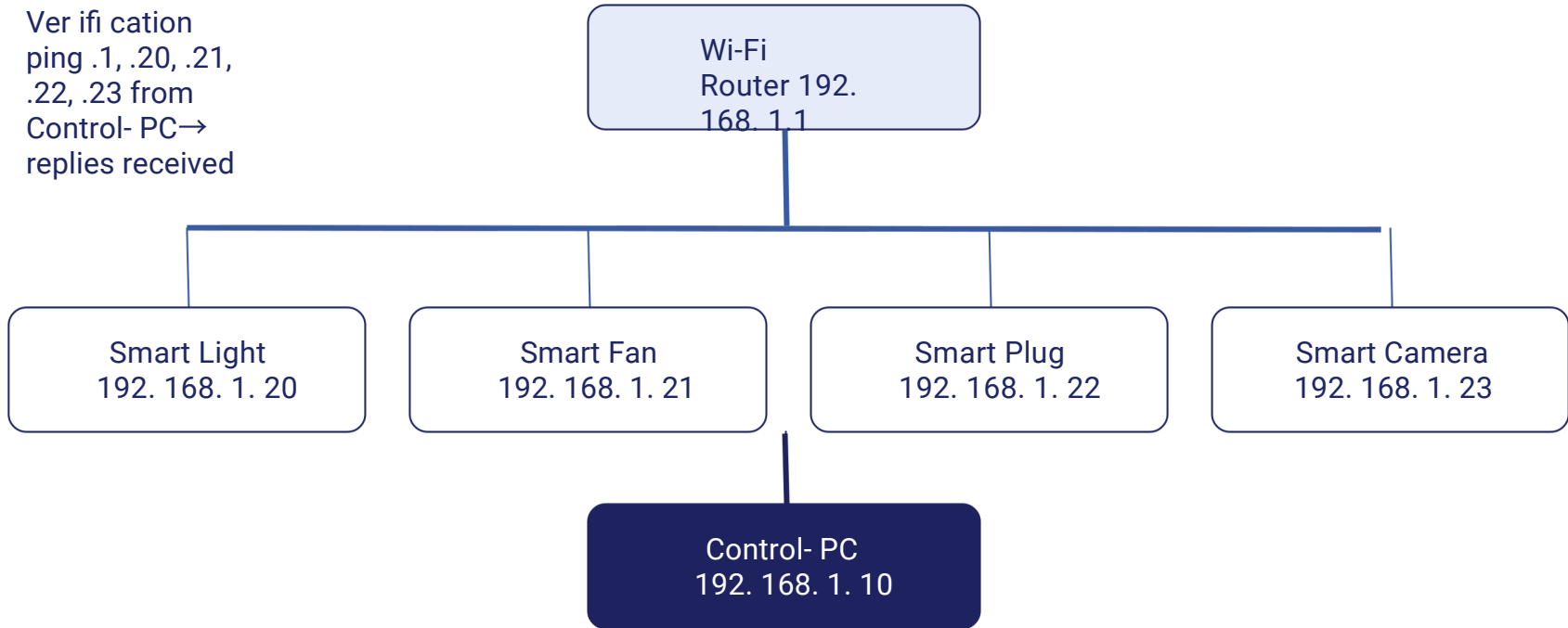
- Capture loopback / Wi-Fi
- Filter tcp.port== 5000
- Commands sent vs captured
- Time command → confirmation
- Avg delay, missed replies

Software /Hardware Functionality

Component	Type	Function in the project
Cisco Packet Tracer	Software (simulator)	Builds and tests the home LAN: router, PC, 4 IoT devices, IP addressing, ping verification
Python 3 (socket)	Software	TCP server holds device states; TCP client sends ON/OFF commands and displays confirmations
Wireshark + Npcap Wi-Fi	Software (analyser)	Captures TCP segments on port 5000; measures delivery delay and counts confirmations
router / home gateway	Simulated hardware	Provides the 192.168.1.0/24 wireless network and default gateway
Control PC / phone	Hardware	Runs the client program – the homeowner's control interface
Smart light, fan, plug, camera	Simulated hardware	End devices that receive commands and change ON/OFF state

Module 1 – Topology Implementation

Verification
ping .1, .20, .21,
.22, .23 from
Control- PC→
replies received



Dashed = wireless association • Subnet mask 255.255.255.0 • Default gateway 192.168.1.1

Module 2 –Control ProgramFlow

- 1 Server binds 127.0.0.1:5000, listens
- 2 Client connects over TCP
- 3 Client sends "DEVICE ON/OFF"
- 4 Server validates device & command
- 5 Server updates state dictionary
- 6 Server sends confirmation string
- 7 Client prints "DEVICE turned ON/OFF"

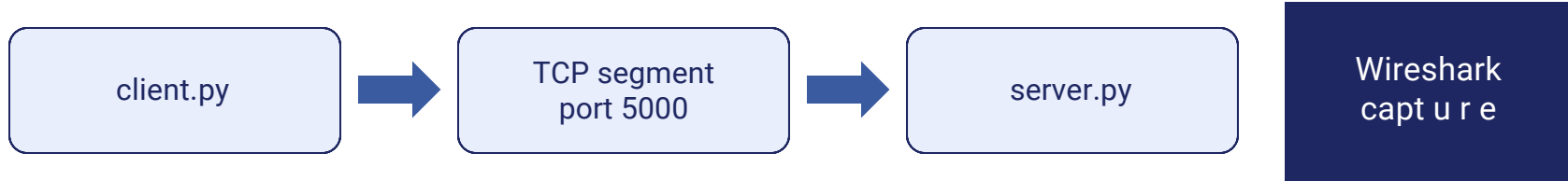
Server responsibilities

- Single device-state dictionary for LIGHT, FAN, PLUG, CAMERA
- Rejects unknown device or command as "Invalid command"
- One confirmation per received command

Client responsibilities

- Menu maps choices 1-4 to device names
- Builds and encodes the command string
- Blocks until the confirmation is received

Module 3 –Verification Method



- Capture is started on the loopback adapter before running the programs (server first, then client).
- Display filter `tcp.port == 5000` isolates only the smart-home control traffic.
- Each command appears as a data segment; each confirmation as the reply segment from the server.
- Delivery delay = timestamp of confirmation packet – timestamp of the matching command packet.
- A command with no reply segment is counted as a missed confirmation.

Result and Comparison

Module	Parameter measured	Result obtained
Module1 Packet Tracer	Reachability of all devices from Control-PC (ping)	[PING RESULT – replace: e.g. 5/5 devices replied]
Module2 Python sockets	Commands sent, confirmations received, success rate	[COMMANDS SENT – replace] / [CONFIRMATIONS – replace] Success rate = [%_ – replace]
Module3 Wireshark	Commands captured, average delivery delay, missed confirmations	Captured [N – replace], avg delay [__ ms – replace], missed [N – replace]

Comparison: the command count recorded by the Python client (Module 2) is compared with the count captured by Wireshark (Module 3). Equal counts with a 100% confirmation rate prove reliable delivery; any difference localises the loss.

Bracketed values are placeholders – fill in with your measured readings.

Limitations and Future Scope

Limitations

- Devices are simulated; no real appliance or relay hardware is switched.
- Client and server run on loopback, so delays are far lower than on a real Wi-Fi link.
- Commands travel as plain text with no authentication or encryption.
- Only one client is supported at a time.
- Packet Tracer IoT behaviour is not driven by the Python program.

Future Scope

- Replace simulated devices with ESP32 / Raspberry Pi relay modules.
- Move to MQTT for publish-subscribe control of many devices.
- Add TLS and user authentication for secure commands.
- Multi-threaded server with device state stored in a database.
- Mobile app UI, scheduling, and voice-assistant integration.
- Automated delay logging instead of manual Wireshark reading.

Use of Ethics and Engineering Standards

Standards Followed

IEEE 802.11 (wireless LAN), IEEE 802.3 (Ethernet), TCP/IP suite (RFC 793, RFC 791), private addressing per RFC 1918.

Capture Ethics

Traffic is captured only on our own loopback/lab interface. No third-party or public network traffic is monitored.

Privacy

The camera device is simulated only – no video or personal data is recorded, stored or transmitted.

Academic Integrity

All code, topology files and captures are produced by the team; external sources are cited in the references.

Safety

No mains wiring or live electrical load is involved, so the work carries no electrical safety risk.

Documentation

Consistent addressing plan, named devices and reproducible test steps allow the work to be re-verified.

Conference /Paper Presentation /Hackathon

- Planned outcome: submission of this work as a short technical paper on verifiable smart-home command delivery.
- Target event: [CONFERENCE / SYMPOSIUM NAME – replace]
- Paper title: [PAPER TITLE – replace]
- Submission / presentation date: [DATE – replace] Status: [submitted / accepted / presented – replace]
- Hackathon participation with this project: [EVENT NAME AND RESULT – replace]
- Proof to be attached: certificate, acceptance mail and presentation photographs.

Replace every bracketed field with your actual event details before the review.

CONCLUSION

The three modules together deliver one working smart home control system. Module 1 establishes a correctly addressed home LAN in Packet Tracer in which all four smart devices are reachable from the control PC. Module 2 implements the control layer as a Python TCP client-server application where every ON/OFF command for the light, fan, plug and camera is answered with an explicit confirmation. Module 3 closes the loop by capturing the same traffic in Wireshark on port 5000, confirming that the commands and confirmations genuinely crossed the network and quantifying the delivery delay and missed confirmations. The project therefore demonstrates not only that a smart home command works, but that it can be proved and measured.

Designed

A reachable
192.168.1.0/24 home LAN

Implemented

Socket-based ON/OFF
control with confirmation

Verified

Packet-level proof and
delay measurement

References

- A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed., Pearson.
- J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, Pearson.
- Cisco Networking Academy, Cisco Packet Tracer – IoT and Wireless Device Documentation.
- Python Software Foundation, "socket – Low-level networking interface," Python 3 Documentation.
- Wireshark Foundation, Wireshark User's Guide – Capture Filters and Time Reference.
- IETF RFC 793, Transmission Control Protocol; IETF RFC 1918, Address Allocation for Private Internets.
- IEEE Std 802.11, Wireless LAN Medium Access Control and Physical Layer Specifications.

Demonstration and Evidence

Live DemoSequence

1. Open the PacketTracerfile and show the topology and IP configuration.
2. Ping all four smart devices from the Control-PC.
3. Start Wireshark capture on the loopback interface.
4. Run server.py, then client.py.
5. Send ON/OFF commands to light, fan, plug and camera.
6. Apply `tcp.port == 5000` and walk through the captured packets and delays.

GPS Photo& VideoEvidence

- Team working photographs with GPS location stamp – [ATTACH – replace]
- Demo video with GPS-tagged metadata – [ATTACH – replace]
- Screenshots: topology, ping results, server terminal, client terminal, Wireshark capture.
- Project folder: Module_1_Packet_Tracer, Module_2_Python, Module_3_Wireshark.